

DISAGRO · Plataforma de Eventos

Resumen técnico de mi prueba Full Stack - Rodrigo Alberto Castellanos Muralles

Para esta prueba construí una aplicación web para confirmar asistencia a un evento, seleccionar un horario y marcar productos o servicios de interés. El frontend calcula una previsualización de promociones, pero el backend vuelve a validar la información antes de guardar la confirmación en MySQL.

Acceso y evaluación

Frontend: <https://fronted-service-production.up.railway.app>

Repositorio: <https://github.com/thetechrod/disagro-prueba-tecnica>

Backend: <https://backend-service-production-0a9c.up.railway.app>

Usuario demo: demo@disagro.com

Contraseña: Disagro2026!

Decisiones principales

- Mantuve todo en un monorepo con frontend, backend, database y docs para facilitar la entrega y la revisión.
- Usé React + TypeScript en frontend, Node.js + Express + TypeScript en backend y MySQL para persistencia.
- Dejé la autenticación como un demo sencillo con JWT. Los usuarios se crean directamente en la base de datos porque no necesitaba agregar un flujo completo de registro.
- Las reglas importantes se validan otra vez en backend. El frontend mejora la experiencia, pero no es la fuente de verdad.
- Guardé snapshots de los ítems confirmados para conservar nombre, tipo y precio, aunque el catálogo cambié después.
- Dejé el proyecto listo para correr con Docker Compose y también lo desplegué en Railway.

Cómo organicé el repositorio

Backend/: API, autenticación, middleware, repositorios y reglas de negocio

Frontend/: interfaz responsive y consumo de la API

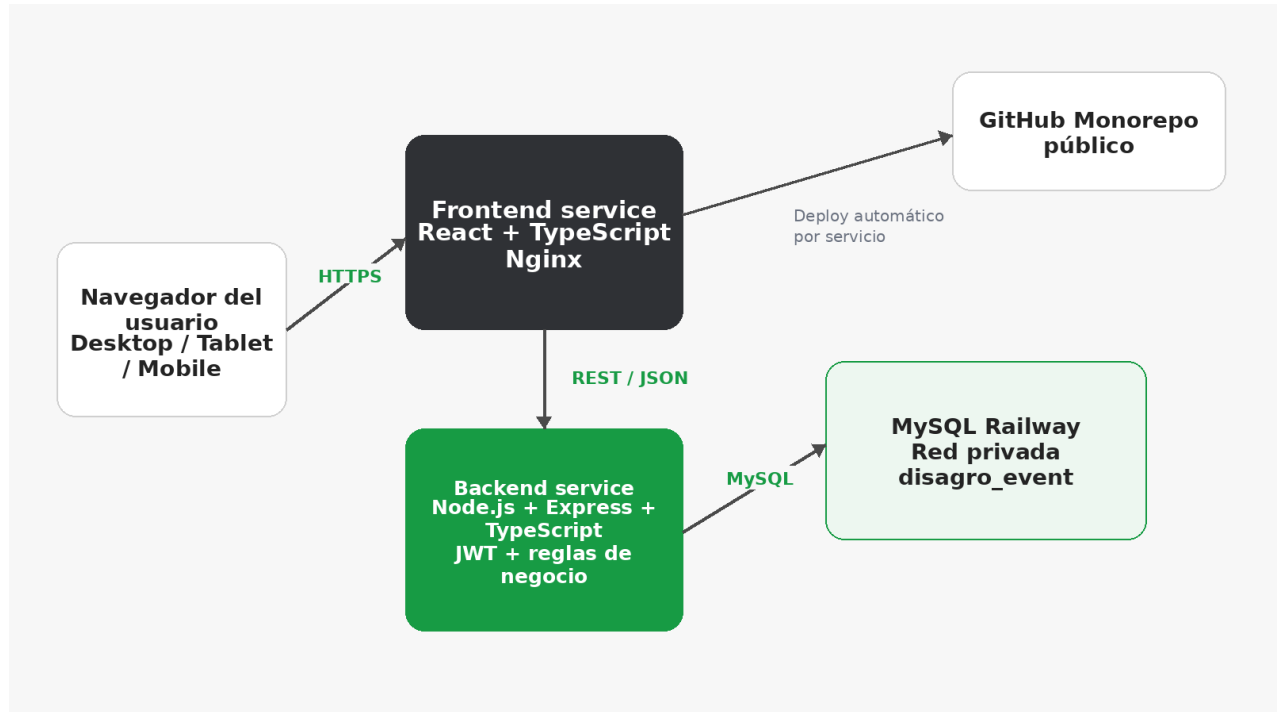
Database/: schema.sql y seed.sql

Docs/: documentación final

docker-compose.yml: stack local completo

Arquitectura de producción

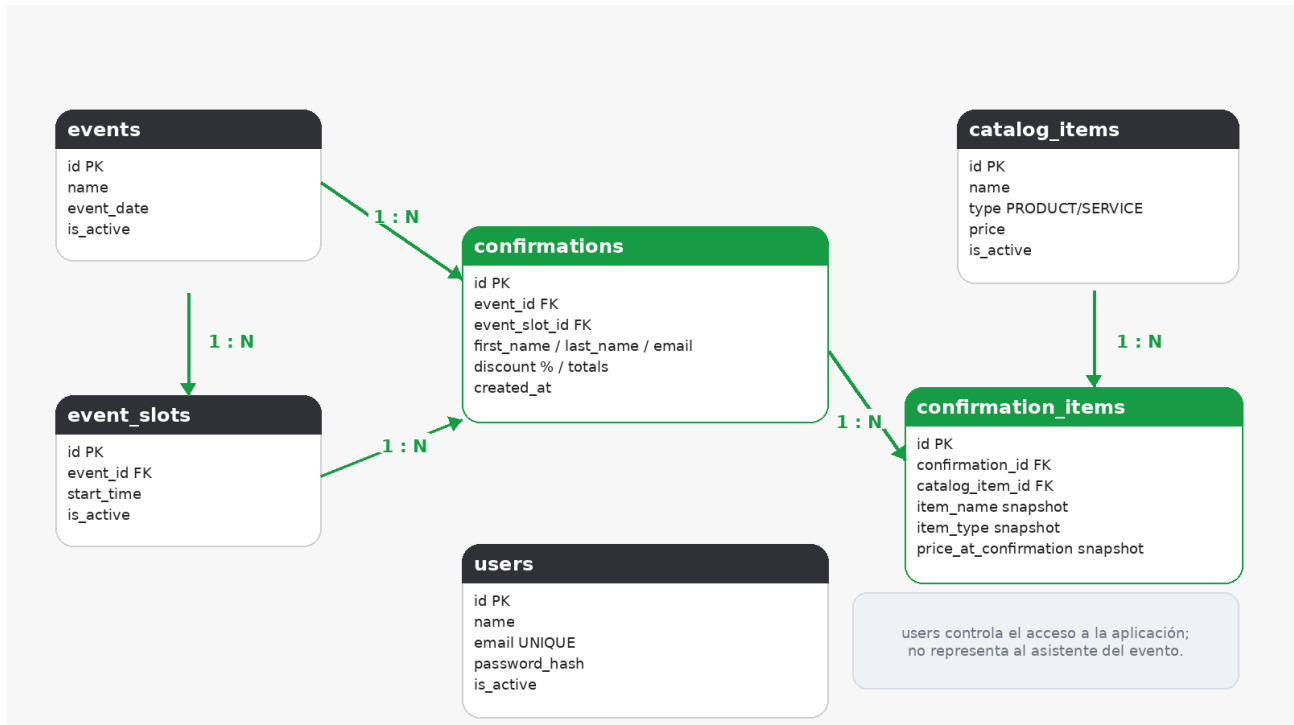
En producción separé el frontend, el backend y MySQL como servicios independientes en Railway. Después de configurar y validar la base, quité el Public Access de MySQL para que el backend se comuniqué con ella únicamente por la red privada de Railway.



- El navegador consume el frontend por HTTPS.
- Nginx sirve el build de React y el frontend consume la API REST del backend.
- El backend concentra autenticación, validaciones y reglas de negocio.
- El backend se conecta a MySQL usando las variables y la red privada de Railway.
- Los cambios del repositorio pueden desplegarse por servicio desde el mismo monorepo.

Modelo de datos

Mantuve el modelo de datos lo más simple posible sin cerrarlo a un solo evento. La idea es que pueda crear otro evento, definir nuevos horarios y reutilizar el mismo flujo sin cambiar la estructura principal de la aplicación.



- **events** me permite reutilizar la plataforma para futuros eventos.
- **event_slots** guarda horarios predefinidos para evitar una fecha u hora libre que luego requiera más validaciones.
- **catalog_items** mantiene productos y servicios en una sola tabla y los diferencia por type.
- **confirmations** guarda los datos del asistente, el horario, los descuentos y los totales de la confirmación.
- **confirmation_items** conserva snapshots del nombre, tipo y precio de cada ítem al momento de confirmar.
- **users** controla únicamente el acceso a la aplicación; no representa a la persona que asiste al evento.

API, reglas de negocio y autenticación

Endpoints principales

POST /api/auth/login — valida las credenciales demo y devuelve un JWT.

GET /api/events/active — devuelve el evento activo, horarios y catálogo.

POST /api/confirmations — valida la solicitud, recalcula promociones y guarda la confirmación.

Validaciones y promociones

- No permito más de una confirmación con el mismo email para el mismo evento.
- Valido que el horario y los ítems existan y estén activos.
- Normalizo IDs repetidos antes de calcular.
- No confío en subtotales o descuentos enviados por el navegador; el backend vuelve a consultar precios y recalcula todo.
- Guardo confirmation y confirmation_items dentro de una transacción para poder hacer rollback si algo falla.
- Probé las ramas de 0%, 3% y 5% de descuento. También agregué un servicio de menor precio para verificar el caso de dos servicios con subtotal menor a Q1,500.

Autenticación del demo

- No implementé registro público porque para esta prueba era suficiente provisionar usuarios directamente en la base de datos.
- La contraseña se guarda con bcrypt + salt, nunca en texto plano.
- Los endpoints protegidos requieren Bearer token mediante middleware en backend.
- Guardo el token en sessionStorage para que la sesión no quede persistida indefinidamente.
- Los secretos y credenciales viven en .env o Railway Variables y no se versionan.
- Si esto fuera producción real, evaluaría refresh tokens, recuperación de contraseña, MFA, rate limiting y rotación de secretos según los requisitos.

Docker, despliegue y pruebas

Ejecución local

- Deje frontend, backend y MySQL dockerizados.
- Docker Compose levanta el stack completo con un solo comando.
- Valide frontend en localhost:8080, backend en localhost:3000 y MySQL Docker desde Windows por localhost:3307.
- schema.sql y seed.sql permiten inicializar una base nueva. También probé docker compose down -v para reconstruirla desde cero.

Despliegue en Railway

- Usé el mismo repositorio para frontend y backend, configurando /frontend y /backend como Root Directory de cada servicio.
- El frontend recibe VITE_API_URL durante el build.
- El backend obtiene la conexión a MySQL mediante variables de Railway.
- Después de inicializar y validar MySQL desde Workbench, eliminé el acceso público y dejé la conexión interna por red privada.

Pruebas que realicé

- Build de frontend y backend sin errores.
- Login correcto e incorrecto, además de endpoints protegidos con y sin Bearer token.
- Confirmación correcta, email duplicado, catalogItemId inválidos y eventSlotId inválido.
- Validación de las diferentes ramas de promociones.
- Snapshot histórico comprobado cambiando temporalmente un precio del catálogo.
- Responsive revisado en desktop, tablet y mobile.
- Flujo completo probado primero en local, después en Docker y finalmente desde las URLs públicas de Railway.

Cierre

Durante la prueba prioricé que el proyecto fuera fácil de entender y de defender: pocos endpoints, reglas claras, una estructura de backend separada sin agregar capas innecesarias y un deployment que se puede reproducir. Si continuara con el proyecto, los siguientes pasos serían aumentar el testing automatizado y endurecer los aspectos de seguridad y observabilidad que dejé fuera del alcance del demo.